

Технологии Программирования

Новая Надежда
(Написание надежного кода)



Технологии программирования
Новая Надежда

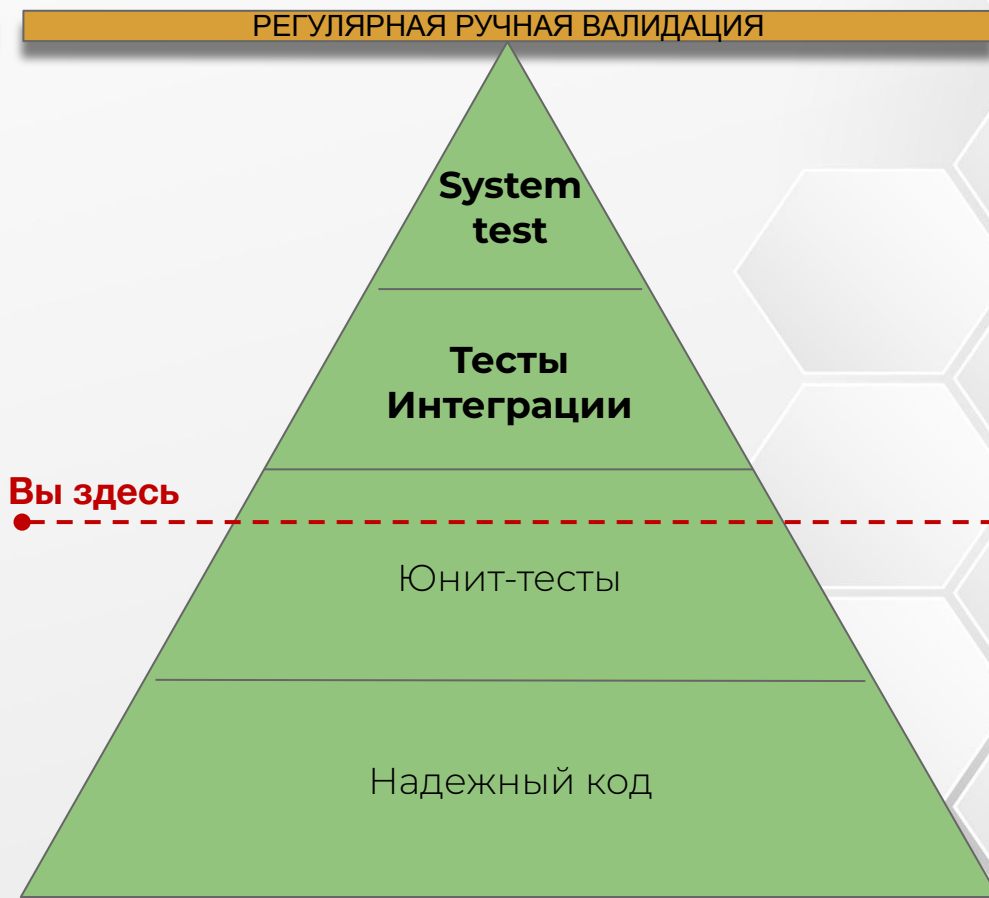
Давным давно...
На прошлой лекции...

Мы познакомились с основами
написания надежного кода.
Как использовать компилятор
для того чтобы избегать
простейших ошибок в коде.

Узнали о методологии TDD,
которая предусматривает
написание юнит-тестов вместе
с основным кодом для того что-
бы повысить надежность кода.

Теперь пришло время познать
ТЕМНУЮ СТОРОНУ разработки
программных продуктов...

Храм знаний



Вопрос!

В чем особенность интеграционных и системных тестов?

Они приводят к удорожанию разработки кода.

Данные тесты влияют позитивно на ROI только в долгосрочной перспективе.

Поэтому их разработка может негативно восприниматься менеджерами и product owner'ами.

Будущее его скрыто от меня в тумане оно.

- Тестирование изменение поведения компонента в зависимости от предыдущих операций,
- Компонент сохраняет состояние от теста к тесту внутри серии,
- Каждый тестовый шаг атомарен, изменения состояния минимальны:

```
SetupTestSuite    //инициализация серии тестов  
Test              //тест меняющий состояние  
Test              //тест меняющий состояние  
Test              //тест меняющий состояние  
TeardownTestSuite
```

- Состояние компонента между тестами не сбрасывается.



Повинуйся воле императора



- Ответвление TDD,
- Общий язык между бизнесом и разработчиками,
- Тесты формируются менеджером продукта или Product owner'ом,
 - Техническое задание в виде теста,
 - Выполнение теста равнозначно выполнению задания.
- Использует человеко-читаемый язык,
 - Стандартная методология Given-When-Then,
 - Реализована в системе Cucumber/Gherkin.
- BDD ограничивает область работы над задачей, сужая зону тестирования и декларируя приемлемые сценарии работы.

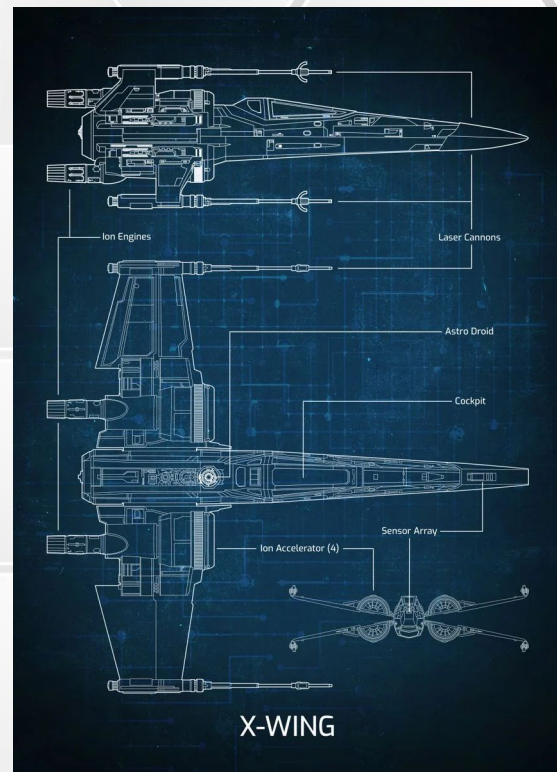
Given-when-then

Given the application is running
 And any valid file is opened
When user click on Delete button
Then application closes the edit screen
 And removes file
 And file does not exists on disk



Познай силу и объедини ее с волей

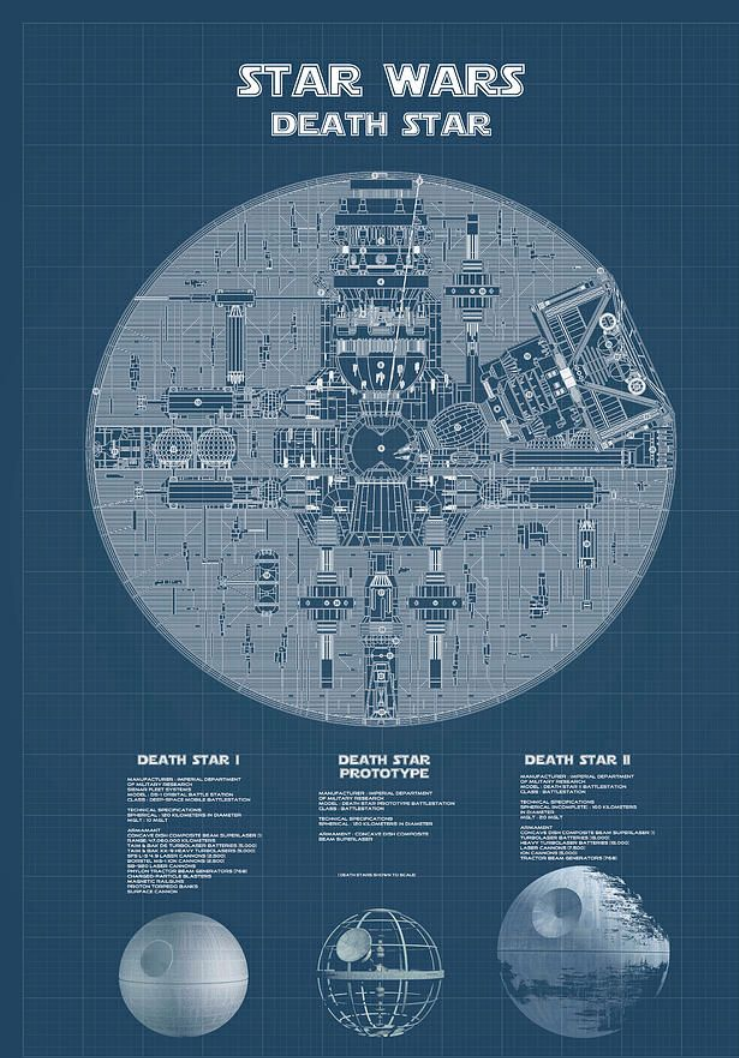
- **Интеграционные тесты** - тесты взаимодействия между компонентами продукта,
- Контроль корректности промежуточных данных на разных стадиях работы программы,
- Исключение лишних модулей для создания контролируемого окружения,
- Проверка цепочки компонентов (метод черного ящика).
 - Чем длиннее цепочка компонент - тем сложнее выявлять причину провала теста,
 - Минимально короткие цепочки,
 - Каскады тестов с увеличением длины цепочек.



Есть в их защите один изъян

- **System test** - проверка поведения продукта в целом
- Работы на стандартных сценариях
 - “Топ100” стандартных сценариев
 - Регрессионное тестирование
 - Сценарии приведшие к дефектам
- Работы в специфических условиях
 - Corner cases
 - Сценарии приведшие к дефектам
 - Высокие нагрузки
 - Недостаток ресурсов (stress test)

В общем случае система автоматического тестирования продукта больше и сложнее чем сам продукт.



Вопрос!

Тесты есть, а дальше что?

Не пуский темную сторону в душу свою

- Интеграция Юнит тестов в процедуру построения продукта
 - Увеличивает локальное время построения продукта
 - Удлиняет процесс разработки
- Автоматическая проверка каждого коммита в репозиторий на ограниченном наборе тестов
 - Все юниты и интеграционные тесты
 - Возможно внедрение системных сценариев
 - Увеличивает время коммита
 - Процедура должна длиться разумное время - не более 45-60 минут
- **Always green.** Тесты этого этапа должны проходить всегда. Падения недопустимы

Может возникнуть соблазн выключить падающие тесты чтобы ускорить разработку. Это прямой путь к Zergrush'у



Бди и днем и ночью

- Автоматизация тестирования продукта,
- Интеграция тестов в CI,
 - Регулярная проверка качества продукта,
 - Чем чаще - тем лучше,
 - Сокращение времени поиска регрессий до интервала между запусками автоматике,
 - Не должно мешать разработке (ночное тестирование).
- Понятные и продуманные логи,
 - Система должна помогать в поиске ошибок,
- Разделенное тестирование больших продуктов.
 - Тестирование основных сценариев ежедневно,
 - Полное тестирование продукта еженедельно.

В худшем случае, поиск ошибок сводится к Git Debugging, а это долгий и нудный процесс поиска коммита который внес ошибку.



Вопрос!

А что можно делать руками?

Тёмная сторона

Тестовые планы - аналог автоматического регулярного тестирования.

Выполняется командой разработки перед релизом
или
QA Командой регулярно

Планы содержат полную информацию о проверке функционала продукта

- Описание теста
- Шаги
- Ожидаемые результаты

Увеличивают ROI на старте или в малых проектах. НО с ростом продукта начинают отвлекать на себя все больше ресурсов.



Исследуя тёмную сторону

Exploratory tests

- Тестирование продукта с точки зрения пользователя,
 - По шагам из документации,
 - По стандартным пользовательским сценариям,
 - Cook book тестирование.
- Слепое тестирование. Без детального знания принципов работы продукта,
 - Новый продукт в знакомой индустрии,
 - Тестирование фич наугад.
- Raccoon testing.
 - Ух-ты! А что это?!?! -_-





THANK YOU!